

```
In [1]: import pandas as pd
from tensorflow.keras.preprocessing.text import Tokenizer
from keras.utils import pad_sequences
from keras.models import Sequential
from keras.layers import Dense, Embedding, LSTM, SpatialDropout1D
from matplotlib import pyplot
from sklearn.model_selection import train_test_split
from keras.utils import to_categorical
import re
from sklearn.wrappers import KerasClassifier
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import GridSearchCV
from keras.layers import Input
from keras.models import load_model
import numpy as np
from tensorflow.keras.preprocessing.sequence import pad_sequences
import tensorflow as tf
```

```
In [2]: # Load dataset
data = pd.read_csv('Sentiment.csv')

# Keeping only the necessary columns
data = data[['text', 'sentiment']]

# Text preprocessing
data['text'] = data['text'].apply(lambda x: x.lower())
data['text'] = data['text'].apply(lambda x: re.sub(r'[^a-zA-Z0-9\s]', '', x))

# Replace 'rt' with a space
for idx, row in data.iterrows():
    data.iloc[idx, 0] = row['text'].replace('rt', ' ')

# Tokenization and padding
max_features = 2000
tokenizer = Tokenizer(num_words=max_features, split=' ')
tokenizer.fit_on_texts(data['text'].values)
X = tokenizer.texts_to_sequences(data['text'].values)
X = pad_sequences(X)
```

```
In [3]: # Tokenization and padding
embed_dim = 128
lstm_out = 196

tokenizer = Tokenizer(num_words=max_features, split=' ')
tokenizer.fit_on_texts(data['text'].values)
X = tokenizer.texts_to_sequences(data['text'].values)
X = pad_sequences(X)

# Model creation with Input Layer
def createmodel():
    model = Sequential()
    model.add(Input(shape=(X.shape[1],)))
    model.add(Embedding(max_features, embed_dim))
    model.add(LSTM(lstm_out, dropout=0.2, recurrent_dropout=0.2))
    model.add(Dense(3, activation='softmax'))
    model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
    return model

# model = createmodel()
# print(model.summary())

# Encoding and splitting the data
labelencoder = LabelEncoder()
integer_encoded = labelencoder.fit_transform(data['sentiment'])
y = to_categorical(integer_encoded)

X_train, X_test, Y_train, Y_test = train_test_split(X, y, test_size=0.33, random_state=42)
```

```
In [4]: # Hyper Tune the model
# Updated KerasClassifier with 'model' instead of 'build_fn'
model = KerasClassifier(model=createmodel, verbose=0)

batch_size = [32]
epochs = [5]
# optimizer = ['SGD', 'RMSprop', 'Adagrad', 'Adam']
# activation = ['softmax', 'relu', 'tanh', 'sigmoid']
# dropout_rate = [0.0, 0.1, 0.2, 0.3, 0.4, 0.5]

param_grid = dict(batch_size=batch_size, epochs=epochs)
grid = GridSearchCV(estimator=model, param_grid=param_grid)
grid_result = grid.fit(X_train, Y_train)
```

```
In [5]: print("Best params : %f using %s"%(grid_result.best_score_,grid_result.best_params_))

Best params : 0.668241 using ('batch_size': 32, 'epochs': 5)
```

```
In [6]: batch_size = grid_result.best_params_['batch_size']
epochs = grid_result.best_params_['epochs']
model = createmodel()
model.fit(X_train, Y_train, epochs = epochs, batch_size=batch_size, verbose = 2)

Epoch 1/5
291/291 - 10s - 35ms/step - accuracy: 0.6404 - loss: 0.8312
Epoch 2/5
291/291 - 9s - 30ms/step - accuracy: 0.7053 - loss: 0.6966
Epoch 3/5
291/291 - 8s - 28ms/step - accuracy: 0.7295 - loss: 0.6291
Epoch 4/5
291/291 - 8s - 29ms/step - accuracy: 0.7562 - loss: 0.5819
Epoch 5/5
291/291 - 9s - 30ms/step - accuracy: 0.7800 - loss: 0.5401

Out[6]: <keras.src.callbacks.history.History at 0x1d7d4561820>
```

```
In [7]: score,acc = model.evaluate(X_test,Y_test,verbose=2,batch_size=batch_size)
print(score)
print(acc)

144/144 - 2s - 15ms/step - accuracy: 0.6785 - loss: 0.8084
0.8083944320678711
0.6784622073173523
```

```
In [8]: # test example case

tweet = "A lot of good things are happening. We are respected again throughout the world, and that's a great thing.@realDonaldTrump"

tweet = tweet.replace('rt',"")

tweet_bg = tokenizer.texts_to_sequences(tweet)

tweet_padded = pad_sequences(tweet_bg)
```

```
In [9]: tweet_sent_pred = model.predict(tweet_padded)

4/4 ----- 0s 82ms/step
```

```
In [10]: tweet_sent_pred
```

```
Out[10]: array([[0.55505395, 0.23779899, 0.2071471 ],
 [0.32727835, 0.29615402, 0.3765676 ],
 [0.32727835, 0.29615402, 0.3765676 ],
 [0.7572455 , 0.19346455, 0.04928995],
 [0.24874997, 0.41596003, 0.33528402],
 [0.32727835, 0.29615402, 0.3765676 ],
 [0.7572455 , 0.19346455, 0.04928995],
 [0.10452683, 0.34663066, 0.54884243],
 [0.32727835, 0.29615402, 0.3765676 ],
 [0.5195091 , 0.21039966, 0.27009124],
 [0.7572455 , 0.19346455, 0.04928995],
 [0.7572455 , 0.19346455, 0.04928995],
 [0.10364561, 0.3030908 , 0.59326357],
 [0.32727835, 0.29615402, 0.3765676 ],
 [0.24874997, 0.41596003, 0.33528402],
 [0.11241672, 0.31100932, 0.576574 ],
 [0.30262294, 0.37865 , 0.31872708],
 [0.12434048, 0.62751855, 0.248149 ],
 [0.5195091 , 0.21039966, 0.27009124],
 [0.23313177, 0.48477122, 0.282097 ],
 [0.32727835, 0.29615402, 0.3765676 ],
 [0.55505395, 0.23779899, 0.2071471 ],
 [0.7648134 , 0.14566073, 0.08952582],
 [0.32727835, 0.29615402, 0.3765676 ],
 [0.32727835, 0.29615402, 0.3765676 ],
 [0.11241672, 0.31100932, 0.576574 ],
 [0.12434048, 0.62751855, 0.24814896],
 [0.30262294, 0.37865 , 0.31872708],
 [0.12434048, 0.62751855, 0.248149 ],
 [0.5195091 , 0.21039966, 0.27009124],
 [0.32727835, 0.29615402, 0.3765676 ],
 [0.32727835, 0.29615402, 0.3765676 ],
 [0.3847201 , 0.50115937, 0.11412058],
 [0.32727835, 0.29615402, 0.3765676 ],
 [0.32727835, 0.29615402, 0.3765676 ],
 [0.55505395, 0.23779899, 0.2071471 ],
 [0.7648134 , 0.14566073, 0.08952582],
 [0.32727835, 0.29615402, 0.3765676 ],
 [0.32727835, 0.29615402, 0.3765676 ],
 [0.12434048, 0.62751855, 0.24814896],
 [0.30262294, 0.37865 , 0.31872708],
 [0.12434048, 0.62751855, 0.248149 ],
 [0.5195091 , 0.21039966, 0.27009124],
 [0.32727835, 0.29615402, 0.3765676 ],
 [0.32727835, 0.29615402, 0.3765676 ],
 [0.3847201 , 0.50115937, 0.11412058],
 [0.7572455 , 0.19346455, 0.04928995],
 [0.7648134 , 0.14566073, 0.08952582],
 [0.7572455 , 0.19346455, 0.04928994],
 [0.42474636, 0.38827172, 0.18698196],
 [0.5195091 , 0.21039966, 0.27009124],
 [0.11241672, 0.31100932, 0.576574 ],
 [0.7572455 , 0.19346455, 0.04928995],
 [0.42474636, 0.38827172, 0.18698196],
 [0.24874997, 0.41596003, 0.33528402],
 [0.32727835, 0.29615402, 0.3765676 ],
 [0.11241672, 0.31100932, 0.576574 ],
 [0.55505395, 0.23779899, 0.2071471 ],
 [0.24874997, 0.41596003, 0.33528402],
 [0.32727835, 0.29615402, 0.3765676 ],
 [0.10364561, 0.3030908 , 0.59326357],
 [0.32727835, 0.29615402, 0.3765676 ],
 [0.55505395, 0.23779899, 0.2071471 ],
 [0.5195091 , 0.21039966, 0.27009124],
 [0.55505395, 0.23779899, 0.2071471 ],
 [0.30262294, 0.37865 , 0.31872708],
 [0.12434048, 0.62751855, 0.248149 ],
 [0.32727835, 0.29615402, 0.3765676 ],
 [0.24874997, 0.41596003, 0.33528402],
 [0.11241672, 0.31100932, 0.576574 ],
 [0.55505395, 0.23779899, 0.2071471 ],
 [0.24874997, 0.41596003, 0.33528402],
 [0.32727835, 0.29615402, 0.3765676 ],
 [0.10364561, 0.3030908 , 0.59326357],
 [0.55505395, 0.23779899, 0.2071471 ],
 [0.12434048, 0.62751855, 0.248149 ],
 [0.10364561, 0.3030908 , 0.59326357],
 [0.7572455 , 0.19346455, 0.04928995],
 [0.12434048, 0.62751855, 0.248149 ],
 [0.55505395, 0.23779899, 0.2071471 ],
 [0.32727835, 0.29615402, 0.3765676 ],
 [0.24874997, 0.41596003, 0.33528402],
 [0.11241672, 0.31100932, 0.576574 ],
 [0.55505395, 0.23779899, 0.2071471 ],
 [0.24874997, 0.41596003, 0.33528402],
 [0.32727835, 0.29615402, 0.3765676 ],
 [0.5195091 , 0.21039967, 0.27009127],
 [0.7648135 , 0.14566073, 0.08952583],
 [0.32727835, 0.29615402, 0.3765676 ],
 [0.55505395, 0.23779899, 0.2071471 ],
 [0.10364561, 0.3030908 , 0.59326357],
 [0.7572455 , 0.19346455, 0.04928995],
 [0.42474636, 0.38827172, 0.18698196],
 [0.39269194, 0.48542523, 0.12188286],
 [0.32727835, 0.29615402, 0.3765676 ]], dtype=float32)
```

```
In [11]: # model.save("model_sent.h5") Legacy : depricated
# Saving model in the recommended .keras format
model.save("model_sent.keras")
```

```
In [12]: # Load the saved model once at the start of the script
model = load_model("model_sent.keras")

# Define the prediction function without @tf.function
def make_prediction(input_text):
    sample_sequence = tokenizer.texts_to_sequences([input_text])
    sample_padded = pad_sequences(sample_sequence, maxlen=X.shape[1])
    predicted_class = model.predict(sample_padded)
    predicted_label = labelencoder.inverse_transform([np.argmax(predicted_class)])
    return predicted_label[0]

# Sample usage
sample_text = "A lot of good things are happening. We are respected again throughout the world, and that's a great thing."
predicted_sentiment = make_prediction(sample_text)

WARNING:tensorflow:5 out of the last 64 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_distributed at 0x00000170733A39C0> triggered tf.function retracing. Tracing is expensive and the excessive number of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors with different shapes, (3) passing Python objects instead of tensors. For (1), please define your @tf.function outside of the loop. For (2), @tf.function has reduce_retracings=True option that can avoid unnecessary retracing. For (3), please refer to https://www.tensorflow.org/guide/function#controlling_retracing and https://www.tensorflow.org/api_docs/python/tf/function for more details.
1/1 ----- 0s 163ms/step

In [13]: print(f"Predicted sentiment: {predicted_sentiment}")

Predicted sentiment: Positive

Video URL:- https://drive.google.com/file/d/1HcrPz2LFx9PL\_KzgtKquUt-gqfAhLL5k/view?usp=sharing
```


